

المفاهيم الأساسية للجودة والوثوقية في نظم المعلومات

إعداد: ا.د. معتصم شفاعمري – كلية هندسة المعلومات والاتصالات- جامعة اليرموك الخاصة

جامعة اليرموك
الخاصة – كلية هندسة
المعلوماتية
والاتصالات

"المفاهيم الأساسية للجودة والوثوقية في نظم المعلومات"

إعداد: ا.د. معتصم شفاعمري –

كلية هندسة المعلوماتية والاتصالات- جامعة اليرموك الخاصة

الفصل الصيفي 2025-2026

Contents

1	"المفاهيم الأساسية للجودة والوثوقية في نظم المعلومات"
1	إعداد: ا.د. معتصم شفاعمري –
1	كلية هندسة المعلوماتية والاتصالات- جامعة اليرموك الخاصة
1	الفصل الصيفي 2025-2026
5	الفصل الأول: مقدمة في جودة البرمجيات والوثوقية
5	1.1 مدخل إلى جودة البرمجيات
5	1.1.1 مفهوم الجودة في سياق البرمجيات
6	1.1.2 الفرق بين جودة المنتج وجودة العملية
6	1.2 مفهوم الوثوقية (Dependability)
6	1.2.1 تعريف الوثوقية وأهميتها
7	1.2.2 العلاقة التكاملية بين الجودة والوثوقية
7	1.3 دروس مستفادة من فشل الأنظمة التاريخية
7	1.3.1 صاروخ أريان 5 (Ariane 5)
8	1.3.2 المركبة الفضائية Mars Climate Orbiter
8	1.3.3 حادثة الخطوط الجوية الكورية رحلة 801
9	1.3.4 جهاز العلاج الإشعاعي - Therac-25
10	الفصل الثاني: ركائز الوثوقية (Attributes of Dependability)
10	2.1 نظرة عامة على ركائز الوثوقية
10	2.2 الاعتمادية (Reliability)
10	2.2.1 تعريف الاعتمادية
11	2.2.2 مقاييس الاعتمادية
11	2.2.3 العوامل المؤثرة على الاعتمادية
12	2.3 التوفر (Availability)
12	2.3.1 تعريف التوفر
12	2.3.2 مقاييس التوفر
13	2.3.3 العلاقة بين الاعتمادية والتوفر
13	2.4 السلامة (Safety)
13	2.4.1 تعريف السلامة

14	2.4.2 الفرق بين السلامة والاعتمادية
14	2.4.3 آليات تحقيق السلامة
15	2.5 النزاهة (Integrity)
15	2.5.1 تعريف النزاهة
15	2.5.2 أبعاد النزاهة
15	2.5.3 آليات تحقيق النزاهة
16	2.6 السرية (Confidentiality)
16	2.6.1 تعريف السرية
16	2.6.2 مستويات السرية
17	2.6.3 آليات تحقيق السرية
17	2.6.4 الفرق بين السرية والنزاهة
17	2.7 قابلية الصيانة (Maintainability)
18	2.7.1 تعريف قابلية الصيانة
18	2.7.2 أبعاد قابلية الصيانة
18	2.7.3 عوامل تحسين قابلية الصيانة
20	الفصل الثالث: مهددات الوثوقية (Threats to Dependability)
20	3.1 نموذج العطل-الخطأ-الفشل (Fault-Error-Failure Model)
20	3.1.1 تعريف النموذج
20	3.1.2 شرح مفصل لكل مكون
22	3.1.3 رسم بياني يوضح العلاقة
22	3.2 تصنيف الأعطال (Classification of Faults)
22	3.2.1 أعطال التدهور (Degradation Faults)
23	3.2.2 أعطال التصميم (Design Faults)
24	3.2.3 الأعطال البيزنطية (Byzantine Faults)
25	3.3 منحنى "حوض الاستحمام" (Bathtub Curve)
25	3.3.1 تعريف المنحنى

الفصل الأول: مقدمة في جودة البرمجيات والوثوقية

1.1 مدخل إلى جودة البرمجيات

1.1.1 مفهوم الجودة في سياق البرمجيات

تُعد جودة البرمجيات (Software Quality) أحد المفاهيم الأساسية في هندسة البرمجيات، وهي تعكس الدرجة التي يلبي بها المنتج البرمجي المتطلبات الموضوع له ويتوافق مع احتياجات وتوقعات المستخدمين. تختلف الجودة في البرمجيات عن الجودة في المنتجات المادية، حيث أن البرمجيات غير ملموسة ولا تخضع للتآكل الفيزيائي بنفس الطريقة التي تخضع لها المنتجات التقليدية.

تعريف الجودة وفقاً لمعيار: IEEE

"الجودة هي الدرجة التي يمتلك بها النظام أو المكون أو العملية الخصائص المطلوبة التي تلبي احتياجات المستخدمين المعبر عنها أو الضمنية".

تعريف الجودة وفقاً لمعيار: ISO 25000

"مدى تلبية المنتج البرمجي للاحتياجات المحددة والضمنية عند استخدامه في ظل ظروف محددة".

أبعاد الجودة البرمجية:

يمكن تقسيم الجودة البرمجية إلى بعدين رئيسيين:

1. **الجودة الوظيفية: (Functional Quality)** وتتعلق بمدى قيام البرمجيات بالوظائف التي صممت من أجلها بشكل صحيح وكامل. وتشمل:

- صحة المخرجات (Correctness)
- اكتمال الوظائف (Completeness)
- دقة المعالجة (Accuracy)

2. **الجودة الهيكلية: (Structural Quality)** وتتعلق بخصائص الكود البرمجي نفسه وبنية النظام، وتشمل:

- قابلية الصيانة (Maintainability)
- قابلية التوسع (Scalability)
- قابلية الاختبار (Testability)
- سهولة الفهم (Understandability)

1.1.2 الفرق بين جودة المنتج وجودة العملية

من المهم التمييز بين مفهومين متكاملين:

جودة العملية (Process Quality)	جودة المنتج (Product Quality)
تركز على الطريقة التي تم بها تطوير المنتج	تركز على خصائص المنتج البرمجي النهائي
تُقاس بمدى فعالية وكفاءة عمليات التطوير	تُقاس بمدى تلبية المتطلبات والمواصفات
أمثلة: الالتزام بالمعايير، استخدام أفضل الممارسات، كفاءة الفريق	أمثلة: الاعتمادية، الأداء، الأمان
يهتم بها فريق التطوير والإدارة	يهتم بها المستخدم النهائي بشكل أساسي

العلاقة بينهما: جودة العملية تؤدي عادةً إلى جودة المنتج، ولكنها ليست ضماناً كاملاً. فوجود عملية تطوير منضبطة يزيد من احتمالية الحصول على منتج عالي الجودة، لكنه لا يمنع الأخطاء بشكل مطلق.

1.2 مفهوم الوثوقية (Dependability)

1.2.1 تعريف الوثوقية وأهميتها

تُعرّف الوثوقية بأنها قدرة النظام على تقديم خدمة يمكن الاعتماد عليها (Dependability)، بحيث يثق المستخدم في أداء النظام وقدرته على القيام بالمهام الموكلة إليه بشكل صحيح ومستمر. وتعتبر الوثوقية صفة شاملة تجمع عدة خصائص أساسية للنظام.

تعريف الوثوقية وفقاً للمعايير الأكاديمية:

"الوثوقية هي صفة النظام التي تسمح للمستخدم بوضع ثقته في الخدمة التي يقدمها النظام بشكل مستمر وموثوق".

أهمية الوثوقية:

تظهر أهمية الوثوقية بشكل خاص في **الأنظمة الحرجة (Critical Systems)** حيث قد يؤدي فشل النظام إلى:

- خسائر بشرية (أنظمة الطيران، الأنظمة الطبية)
- خسائر مالية فادحة (الأنظمة المصرفية، أنظمة التداول)
- أضرار بيئية (أنظمة التحكم في المحطات النووية)
- تأثيرات اجتماعية واسعة (أنظمة الاتصالات، شبكات الطاقة)

1.2.2 العلاقة التكاملية بين الجودة والوثوقية

يمكن فهم العلاقة بين الجودة والوثوقية من خلال النقاط التالية:

1. **الوثوقية هي أحد أبعاد الجودة:** تعتبر الوثوقية مكوناً أساسياً من مكونات الجودة البرمجية، حيث لا يمكن اعتبار نظام برمجي عالي الجودة إذا كان غير موثوق.
2. **الجودة شرط أساسي للوثوقية:** لا يمكن تحقيق نظام موثوق دون الالتزام بمعايير الجودة في جميع مراحل التطوير.
3. **الوثوقية تتجاوز الجودة الوظيفية:** قد يكون النظام صحيحاً وظيفياً ولكنه غير موثوق بسبب عدم قدرته على التعامل مع الأعطال أو الظروف غير المتوقعة.
4. **التكامل في القياس:** يمكن قياس الجودة والوثوقية باستخدام مقاييس متداخلة، مثل متوسط الوقت بين الأعطال (MTBF) ونسبة توفر النظام.

1.3 دروس مستفادة من فشل الأنظمة التاريخية

1.3.1 صاروخ أريان 5 (Ariane 5)

التاريخ 4 يونيو 1996

الوصف: انفجر صاروخ أريان 5 الأوروبي بعد 37 ثانية فقط من إطلاقه، في رحلته الأولى.

السبب التقني:

- تم إعادة استخدام كود برمجي من صاروخ أريان 4 دون إعادة اختبار كافٍ.
- سبب الفشل هو تجاوز قيمة متغير (Overflow) في نظام الملاحة بالقصور الذاتي.

- كان المتغير من النوع 16 بت، بينما القيمة المحسوبة كانت تتطلب 64 بت.
- أدى تجاوز القيمة إلى إرسال بيانات خاطئة إلى نظام التحكم، مما تسبب في انحراف الصاروخ وتدميره.

الدروس المستفادة:

- ضرورة إعادة اختبار الكود في سياقه الجديد حتى لو كان يعمل بشكل صحيح في النظام السابق.
- أهمية استخدام معالجة استثنائية قوية. (Exception Handling)
- ضرورة مراجعة الافتراضات التصميمية عند تغيير بيئة التشغيل.
- تكلفة فشل النظام البرمجي قد تكون باهظة جداً (حوالي 370 مليون دولار).

1.3.2 المركبة الفضائية Mars Climate Orbiter

التاريخ 23: سبتمبر 1999

الوصف: فقدت وكالة ناسا الاتصال بالمركبة الفضائية عند دخولها مدار المريخ.

السبب التقني:

- خلط بين وحدات القياس: استخدم فريق ناسا وحدات المترية (Metric) بينما استخدم فريق آخر وحدات الإمبراطورية (Imperial).
- لم يتم تحويل قيم الدفع من نظام الوحدات البريطاني (رطل-ثانية) إلى النظام المتري (نيوتن-ثانية).
- أدى ذلك إلى انخفاض ارتفاع المركبة عن المسار المخطط له، مما تسبب في احتراقها في الغلاف الجوي للمريخ.

الدروس المستفادة:

- أهمية التوحيد القياسي لوحدات القياس في المشاريع التعاونية.
- ضرورة التحقق من صحة البيانات بين الأنظمة الفرعية المختلفة.
- أهمية وجود آليات للتحقق من الاتساق بين المكونات المختلفة.
- تكلفة الإهمال في تفاصيل التكامل (تكلفة المركبة: 125 مليون دولار).

1.3.3 حادثة الخطوط الجوية الكورية رحلة 801

التاريخ 6: أغسطس 1997

الوصف: تحطمت طائرة بوينغ 747-300 التابعة للخطوط الجوية الكورية أثناء محاولتها الهبوط في مطار أنطونيو ب. وان بات في غوام.

السبب التقني:

- كان نظام الإنذار بالارتفاع (GPWS - Ground Proximity Warning System) في الطائرة معطلاً، لكن الطاقم لم يكن على علم بذلك.
- لم يتم الإبلاغ عن عطل النظام في سجلات الصيانة بشكل صحيح.
- اعتماد الطاقم على معلومات غير دقيقة عن موقع الطائرة.

الدروس المستفادة:

- أهمية أنظمة الكشف عن الأعطال وإبلاغ المستخدم بها.
- ضرورة وجود عمليات صيانة وإبلاغ دقيقة عن حالة الأنظمة.
- أهمية تدريب المستخدمين على التعامل مع حالات فشل الأنظمة.
- تأثير فشل النظام البرمجي (أو عطله) يمكن أن يكون كارثياً على الأرواح (وفاة 228 شخصاً).

1.3.4 جهاز العلاج الإشعاعي - Therac-25

التاريخ: 1985-1987 :

الوصف: جهاز Therac-25 هو جهاز علاج إشعاعي استُخدم في المستشفيات، وتسبب في جرعات زائدة من الإشعاع لعدة مرضى، مما أدى إلى وفيات وإصابات خطيرة.

السبب التقني:

- تم إعادة استخدام كود من إصدارات سابقة (Therac-6) و (Therac-20) دون مراعاة الاختلافات في الأجهزة.
- لم تكن هناك آليات أمان كافية (Interlocks) للكشف عن الأخطاء في الجرعة.
- عدم وجود واجهة مستخدم واضحة تبلغ المشغل بحدوث خطأ.
- كان هناك عطل برمجي في تزامن العمليات (Race Condition) يسمح بتعديل الجرعة بشكل غير صحيح.

الدروس المستفادة:

- ضرورة تصميم أنظمة أمان متعددة المستويات في الأنظمة الحرجة.
- أهمية اختبار النظام ككل، وليس فقط المكونات المنفصلة.
- يجب أن تكون واجهة المستخدم مصممة لتوعية المشغل بحالة النظام ومخاطره.
- الأنظمة التي تتفاعل مع البشر تحتاج إلى اعتبارات خاصة للسلامة.

الفصل الثاني: ركائز الوثوقية (Attributes of Dependability)

2.1 نظرة عامة على ركائز الوثوقية

تعتمد الوثوقية على مجموعة من الصفات أو الركائز الأساسية التي يجب أن تتوفر في النظام. يمكن تصنيف هذه الركائز إلى ثلاث فئات رئيسية:

1. صفات متعلقة بجودة الخدمة: الاعتمادية، التوفر، قابلية الصيانة.
2. صفات متعلقة بالأمن: السرية، النزاهة.
3. صفات متعلقة بالسلامة: السلامة.

العلاقة بين الركائز:

- هناك تداخل بين هذه الصفات، حيث أن تحسين إحداها قد يؤثر على الأخرى (أحياناً إيجاباً وأحياناً سلباً).
- مثال: تحسين الأمان (زيادة التعقيد) قد يؤثر سلباً على الأداء.
- مثال: زيادة الاعتمادية (من خلال التكرار) تحسن التوفر أيضاً.

2.2 الاعتمادية (Reliability)

2.2.1 تعريف الاعتمادية

الاعتمادية (Reliability) هي قدرة النظام على العمل بشكل متواصل دون أخطاء أو أعطال خلال فترة زمنية محددة وفي ظل ظروف تشغيل معينة.

تعريف: IEEE

"الاعتمادية هي قدرة النظام أو المكون على أداء الوظائف المطلوبة في ظل ظروف محددة لفترة زمنية محددة".

2.2.2 مقاييس الاعتمادية

1. MTBF - متوسط الوقت بين الأعطال: (Mean Time Between Failures)

- هو متوسط الفترة الزمنية بين فشليين متتاليين في النظام.
- يستخدم للأنظمة القابلة للإصلاح.
- الصيغة: $MTBF = \text{مجموع أوقات التشغيل} \div \text{عدد الأعطال}$

2. MTTF - متوسط الوقت حتى الفشل: (Mean Time To Failure)

- هو متوسط الفترة الزمنية المتوقعة حتى حدوث الفشل الأول.
- يستخدم للأنظمة غير القابلة للإصلاح.
- الصيغة: $MTTF = \text{مجموع أوقات التشغيل حتى الفشل} \div \text{عدد الأنظمة المختبرة}$

3. معدل الفشل: (Failure Rate - λ)

- هو عدد الأعطال المتوقعة في وحدة الزمن.
- الصيغة: $\lambda = 1 \div MTTF$

4. احتمالية البقاء: (Reliability Function - $R(t)$)

- هي احتمال أن يعمل النظام بشكل صحيح حتى الزمن t .
- في حالة التوزيع الأسي: $R(t) = e^{(-\lambda t)}$

2.2.3 العوامل المؤثرة على الاعتمادية

العامل	التأثير
تعقيد النظام	كلما زاد التعقيد، زادت احتمالية الأعطال
جودة التصميم	التصميم الجيد يقلل من الأعطال
بيئة التشغيل	الظروف القاسية تزيد من الأعطال
عمر النظام	قد تزيد الأعطال مع تقدم العمر (للمكونات المادية)

2.3 التوفر (Availability)

2.3.1 تعريف التوفر

التوفر (Availability) هو استعداد النظام لتقديم الخدمة عند الطلب في أي لحظة زمنية. يعكس التوفر مدى قدرة النظام على أن يكون في حالة تشغيل صحيحة عندما يحتاجه المستخدم.

تعريف: IEEE

"التوفر هو الدرجة التي يكون بها النظام في حالة تشغيل وقابل للوصول عند الطلب للاستخدام".

2.3.2 مقاييس التوفر

1. التوفر النسبي: (Availability Ratio)

- النسبة المئوية للوقت الذي يكون فيه النظام متاحاً.
- الصيغة: التوفر = $(\text{وقت التشغيل الكلي}) \div (\text{وقت التشغيل الكلي} + \text{وقت التوقف}) \times 100\%$

2. التوفر المطلق: (Uptime)

- الفترة الزمنية التي يكون فيها النظام متاحاً ومستعداً لتقديم الخدمة.

3. وقت التوقف: (Downtime)

- الفترة الزمنية التي يكون فيها النظام غير متاح.

4. تصنيف التوفر حسب النسبة المئوية:

الوصف	وقت التوقف السنوي	مستوى التوفر
توفر منخفض	36.5 يوم	90% ("أربعة تسعات")
توفر متوسط	3.65 يوم	99% ("تسعتان")
توفر مرتفع	8.76 ساعة	99.9% ("ثلاث تسعات")
توفر عالٍ جداً	52.56 دقيقة	99.99% ("أربع تسعات")
توفر شبه كامل	5.26 دقيقة	99.999% ("خمس تسعات")

توفر شبه كامل جداً	31.5 ثانية	99.9999% ("ست تسعات")
--------------------	------------	-----------------------

2.3.3 العلاقة بين الاعتمادية والتوفر

يمكن التعبير عن التوفر بدلالة الاعتمادية (MTBF) ووقت الإصلاح: (MTTR - Mean Time To Repair)

$$\text{التوفر} = \text{MTBF} \div (\text{MTBF} + \text{MTTR})$$

حيث:

- **MTBF:** متوسط الوقت بين الأعطال (مقياس للاعتمادية)
- **MTTR:** متوسط الوقت اللازم لإصلاح النظام بعد العطل

الاستنتاجات:

- تحسين الاعتمادية (زيادة MTBF) يزيد التوفر.
- تقليل وقت الإصلاح (تقليل MTTR) يزيد التوفر أيضاً.
- في الأنظمة الحرجة، يجب تحسين كلا العاملين.

مثال: نظام لديه MTBF = 1000 ساعة و MTTR = 10 ساعات

- التوفر = $1000 \div (1000 + 10) = 0.9901 = 99.01\%$

2.4 السلامة (Safety)

2.4.1 تعريف السلامة

السلامة (Safety) هي قدرة النظام على تجنب وقوع أعطال كارثية تؤدي إلى إصابات بشرية أو خسائر مالية فادحة أو أضرار بيئية. تركز السلامة على منع العواقب الوخيمة حتى في حالة حدوث فشل.

تعريف: IEEE

"السلامة هي قدرة النظام على عدم التسبب في حالات خطيرة أو أضرار للأشخاص أو البيئة أو الممتلكات".

2.4.2 الفرق بين السلامة والاعتمادية

الاعتمادية (Reliability)	السلامة (Safety)
تركز على تكرار الفشل	تركز على العواقب وليس على تكرار الفشل
تتعامل مع جميع أنواع الأعطال	تتعامل مع الأعطال الكارثية
بعض الأعطال قد تكون مقبولة	حتى فشل واحد قد يكون غير مقبول
مثال: نظام الترفيه في السيارة	مثال: نظام الفرامل في السيارة

2.4.3 آليات تحقيق السلامة

1. تصميم آمن أساساً: (Safety by Design)

- استخدام مبادئ التصميم التي تجعل النظام آمناً بطبيعته.
- مثال: استخدام نظام الفرامل الاحتياطي في المصاعد.

2. التكرار والتنوع: (Redundancy & Diversity)

- وجود مكونات متعددة تؤدي نفس الوظيفة بشكل مستقل.
- إذا فشل أحد المكونات، يتولى الآخر المهمة.

3. آليات التوقف الآمن: (Fail-Safe Mechanisms)

- تصميم النظام للوصول إلى حالة آمنة عند حدوث فشل.
- مثال: قضبان السكك الحديدية التي تقطع التيار الكهربائي عند اكتشاف عطل.

4. مراقبة السلامة: (Safety Monitoring)

- أنظمة تراقب باستمرار حالة النظام وتتخذ إجراءات وقائية.
- مثال: أنظمة الكشف عن الدخان والحرائق.

5. التحديثات الأمنية: (Safety Updates)

- تحديث النظام باستمرار لمعالجة الثغرات المكتشفة.

2.5 النزاهة (Integrity)

2.5.1 تعريف النزاهة

النزاهة (Integrity) هي صفة النظام التي تضمن منع التعديل غير المصرح به للبيانات أو البرمجيات، وضمان صحة المعلومات واتساقها وحمايتها من التلف أو الفساد أو التغيير غير المشروع.

تعريف: ISO/IEC 25010

"النزاهة هي الدرجة التي يحمي بها النظام أو المنتج البيانات أو المعلومات من التعديل أو التدمير غير المصرح به".

2.5.2 أبعاد النزاهة

1. نزاهة البيانات: (Data Integrity)

- ضمان أن البيانات لم يتم تغييرها بشكل غير مصرح به.
- الحماية من الفساد، التلف، أو التعديل غير المشروع.

2. نزاهة النظام: (System Integrity)

- ضمان أن النظام يعمل كما هو متوقع دون تدخل غير مصرح به.
- الحماية من هجمات القرصنة أو البرمجيات الخبيثة.

3. نزاهة المصدر: (Source Integrity)

- ضمان أن المعلومات تأتي من مصدر موثوق ولم يتم التلاعب بها.
- استخدام التوقيعات الرقمية وشهادات المصادقة.

2.5.3 آليات تحقيق النزاهة

1. آليات التحقق: (Verification Mechanisms)

- المجاميع الاختبارية (Checksums)
- التواقيع الرقمية (Digital Signatures)
- دوال التجزئة (Hash Functions)

2. آليات التحكم بالوصول: (Access Control)

- تحديد من يمكنه تعديل البيانات.
- تحديد مستويات الصلاحيات المختلفة.

3. آليات التوثيق: (Logging)

- تسجيل جميع عمليات التعديل.
- تتبع مصدر التغييرات.

4. آليات التدقيق: (Auditing)

- مراجعة السجلات للتأكد من النزاهة.
- اكتشاف أي محاولات تعديل غير مشروعة.

2.6 السرية (Confidentiality)

2.6.1 تعريف السرية

السرية (Confidentiality) هي صفة النظام التي تضمن منع الكشف غير المصرح به للمعلومات. تعني أن البيانات والمعلومات لا يمكن الوصول إليها أو قراءتها إلا من قبل الأشخاص أو الأنظمة المخولة بذلك.

تعريف: ISO/IEC 25010

"السرية هي الدرجة التي يضمن بها النظام أو المنتج أن البيانات لا يمكن الوصول إليها أو قراءتها إلا من قبل المستخدمين المصرح لهم".

2.6.2 مستويات السرية

1. السرية المطلقة: (Absolute Confidentiality)

- لا يمكن لأي شخص غير مخول الوصول إلى البيانات بأي شكل.

2. السرية المشروطة: (Conditional Confidentiality)

- يمكن الوصول للبيانات في ظروف معينة وبإذن خاص.

3. السرية الجزئية: (Partial Confidentiality)

- أجزاء معينة من البيانات محمية بينما أجزاء أخرى عامة.

2.6.3 آليات تحقيق السرية

1. التشفير: (Encryption)

- تشفير البيانات أثناء التخزين (Encryption at Rest)
- تشفير البيانات أثناء النقل (Encryption in Transit)

2. آليات التحكم بالوصول: (Access Control)

- تحديد من يمكنه قراءة البيانات.
- استخدام المصادقة متعددة العوامل (MFA).

3. إخفاء البيانات: (Data Masking)

- إخفاء أجزاء من البيانات الحساسة.
- مثال: إخفاء أرقام بطاقات الائتمان جزئياً.

4. التقسيم الأمني: (Security Segmentation)

- تقسيم النظام إلى مناطق أمنية مختلفة.
- كل منطقة لها مستوى حماية خاص بها.

2.6.4 الفرق بين السرية والنزاهة

النزاهة (Integrity)	السرية (Confidentiality)
تمنع التعديل غير المصرح به	تمنع الكشف غير المصرح به
تتعامل مع "من يمكنه تغيير البيانات"	تتعامل مع "من يمكنه رؤية البيانات"
مثال: التوقيع الرقمي على البريد الإلكتروني	مثال: تشفير البريد الإلكتروني
تنتهك بتغيير البيانات	تنتهك بقراءة البيانات

2.7 قابلية الصيانة (Maintainability)

2.7.1 تعريف قابلية الصيانة

قابلية الصيانة (Maintainability) هي سهولة إصلاح النظام وتطويره وتحديثه وإعادة تكوينه واستيعابه للتغيرات. تعبر عن الجهد المطلوب لإجراء التعديلات اللازمة على النظام.

تعريف: IEEE

"قابلية الصيانة هي سهولة إجراء التعديلات على نظام أو مكون لتغيير أو إصلاح أو تحسين النظام أو مكوناته".

2.7.2 أبعاد قابلية الصيانة

1. قابلية التحليل: (Analyzability)

- سهولة فهم هيكل النظام وتتبع الأخطاء.

2. قابلية التغيير: (Changeability)

- سهولة إجراء التعديلات على النظام.

3. الاستقرار: (Stability)

- مقاومة النظام للتغيرات غير المتوقعة عند إجراء التعديلات.

4. قابلية الاختبار: (Testability)

- سهولة اختبار النظام بعد إجراء التعديلات.

2.7.3 عوامل تحسين قابلية الصيانة

1. تصميم نمطي: (Modular Design)

- تقسيم النظام إلى وحدات مستقلة.
- سهولة استبدال أو تحديث كل وحدة.

2. توثيق جيد: (Good Documentation)

- وصف واضح لهيكل النظام ووظائفه.
- شرح التغييرات والإصدارات.

3. معايير برمجية: (Coding Standards)

- كتابة كود نظيف ومنظم.

○ استخدام أسماء دالة ومعرفات واضحة.

4. أتمتة الاختبار: (Test Automation)

○ اختبارات آلية للتحقق من عدم كسر النظام.

○ اختبارات انحدار (Regression Tests) بعد كل تغيير.

5. إدارة التهيئة: (Configuration Management)

○ تتبع جميع التغييرات بشكل منظم.

○ التحكم في الإصدارات. (Version Control)

الفصل الثالث: مهددات الوثوقية (Threats to Dependability)

3.1 نموذج العطل-الخطأ-الفشل (Fault-Error-Failure Model)

3.1.1 تعريف النموذج

يعد نموذج العطل-الخطأ-الفشل (Fault-Error-Failure) نموذجاً أساسياً لفهم كيفية حدوث الأعطال في الأنظمة الحاسوبية. يصف النموذج سلسلة الأحداث التي تؤدي من وجود عيب داخلي في النظام إلى فشل ملاحظ من قبل المستخدم.

مكونات النموذج:

1. العطل (Fault): عيب أو خلل داخلي في النظام.
2. الخطأ (Error): حالة غير صحيحة للنظام تنشأ عن تنشيط العطل.
3. الفشل (Failure): انحراف الخدمة المقدمة عن السلوك المطلوب (ما يلاحظه المستخدم).

3.1.2 شرح مفصل لكل مكون

أ. العطل (Fault):

التعريف: العطل هو عيب أو خلل في مكون من مكونات النظام. يمكن أن يكون العطل:

- عطلاً برمجياً (Software Fault): خطأ في الكود، التصميم، أو المواصفات.
- عطلاً مادياً (Hardware Fault): عيب في المكونات المادية مثل الذاكرة، المعالج، القرص الصلب.
- عطلاً بشرياً (Human Fault): خطأ في التفاعل مع النظام، أو في عمليات التشغيل والصيانة.

خصائص العطل:

- العطل قد يكون موجوداً لفترة طويلة دون أن يكتشف.
- العطل قد لا يسبب أي مشكلة إلا عند تنشيطه بظروف معينة.
- العطل قد يكون عابراً (موقتاً) أو دائماً.

أمثلة على الأعطال:

- وجود خطأ في خوارزمية حساب الفائدة في نظام مصرفي.
- وجود قطاع تالف في القرص الصلب.
- خطأ في إدخال البيانات من قبل المستخدم.

ب. الخطأ (Error)

التعريف: الخطأ هو حالة غير صحيحة للنظام تنشأ عن تنشيط العطل. عند تنشيط العطل، يصبح النظام في حالة خاطئة.

خصائص الخطأ:

- الخطأ حالة داخلية للنظام قد لا تكون مرئية للمستخدم.
- الخطأ قد ينتشر عبر النظام ويؤثر على مكونات أخرى.
- الخطأ قد يكون قابلاً للتصحيح قبل أن يتحول إلى فشل.

أمثلة على الأخطاء:

- قيمة خاطئة في متغير بسبب خطأ في الحساب.
- حالة غير صحيحة في قاعدة البيانات بسبب عملية غير مكتملة.
- استثناء (Exception) في البرنامج بسبب عملية غير مدعومة.

ج. الفشل (Failure)

التعريف: الفشل هو انحراف الخدمة المقدمة عن السلوك المطلوب. الفشل هو ما يلاحظه المستخدم النهائي للنظام.

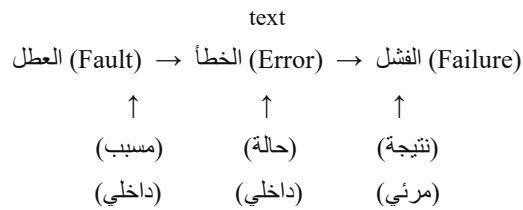
خصائص الفشل:

- الفشل هو النتيجة النهائية لسلسلة العطل-الخطأ-الفشل.
- الفشل قد يكون كاملاً (توقف النظام) أو جزئياً (تدهور الأداء).
- الفشل قد يكون مؤقتاً أو دائماً.

أمثلة على الفشل:

- تعطل التطبيق وإغلاقه فجأة.
- إعطاء نتائج غير صحيحة للمستخدم.
- عدم استجابة النظام لأوامر المستخدم.

3.1.3 رسم بياني يوضح العلاقة



مثال تطبيقي:

1. **العطل:** خطأ برمجي في دالة حساب المتوسط حيث لا يتم التعامل مع القسمة على صفر.
2. **الخطأ:** عندما يطلب المستخدم حساب متوسط أرقام تشمل قيمة صفر في المقام، تنتج قيمة غير معرفة (NaN) في المتغير.
3. **الفشل:** يظهر تطبيق الحاسبة رسالة خطأ أو يتعطل، ويشعر المستخدم بأن التطبيق غير موثوق.

3.2 تصنيف الأعطال (Classification of Faults)

3.2.1 أعطال التدهور (Degradation Faults)

التعريف: أعطال التدهور هي أعطال تحدث في المكونات المادية للنظام وتتطور مع مرور الزمن نتيجة للاستخدام والتآكل الطبيعي.

خصائص أعطال التدهور:

- تحدث في المكونات المادية. (Hardware)
- تظهر تدريجياً مع تقدم العمر.

- تتبع منحنى "حوض الاستحمام" (Bathtub Curve).
- يمكن التنبؤ بها نوعاً ما باستخدام النماذج الإحصائية.

أمثلة على أعطال التدهور:

- تلف الأقراص الصلبة بعد سنوات من الاستخدام.
- تآكل الذاكرة الفلاشية بعد عدد معين من عمليات الكتابة.
- ارتفاع درجة حرارة المعالج بسبب تراكم الغبار.
- ضعف بطارية الأجهزة المحمولة مع مرور الزمن.

3.2.2 أعطال التصميم (Design Faults)

التعريف: أعطال التصميم هي أخطاء بشرية تحدث أثناء مرحلة التصميم أو التطوير أو المواصفات. هذه الأعطال خاصة بالبرمجيات (ولكنها قد تحدث أيضاً في التصميم المادي).

خصائص أعطال التصميم:

- تحدث في البرمجيات بشكل أساسي.
- لا تتبع منحنى حوض الاستحمام (لأن البرمجيات لا تتآكل).
- موجودة منذ لحظة كتابة الكود.
- تظهر عند تنشيطها بظروف معينة (مثل بيانات إدخال محددة).

أمثلة على أعطال التصميم:

- خطأ في خوارزمية الفرز يؤدي إلى نتائج غير صحيحة.
- عدم معالجة حالة استثنائية (مثل إدخال نص بدلاً من رقم).
- خطأ في منطق العمل (Business Logic) يؤدي إلى حسابات خاطئة.
- ثغرة أمنية في التصميم تسمح باختراق النظام.

لماذا لا تتبع أعطال التصميم منحنى حوض الاستحمام؟

- البرمجيات لا تتآكل مع الزمن.
- العطل البرمجي موجود منذ لحظة كتابته.
- ظهور العطل يعتمد على تنشيطه وليس على عمر النظام.

- اكتشاف الأعطال البرمجية قد يزيد مع زيادة الاستخدام (وليس العكس).

3.2.3 الأعطال البيزنطية (Byzantine Faults)

التعريف: الأعطال البيزنطية هي أعطال حيث يتصرف المكون بشكل عشوائي أو خبيث دون أن يكون هناك نمط واضح، مما يصعب اكتشاف الخطأ وتحديد مصدره. سميت بهذا الاسم نسبة إلى مشكلة "الجنرالات البيزنطيين (Byzantine Generals Problem)".

خصائص الأعطال البيزنطية:

- قد يكون المكون معطلاً بشكل كامل أو يتصرف بشكل غير متوقع.
- قد يكون المكون مخترقاً ويتصرف بشكل خبيث.
- قد يُعطي المكون معلومات مضللة لأنظمة أخرى.
- صعوبة اكتشاف المصدر الحقيقي للخطأ.

أمثلة على الأعطال البيزنطية:

- هجمات رفض الخدمة الموزعة (DDoS).
- اختراق نظام وتحويله لتزويد المعلومات الخاطئة.
- عطل في مستشعر يعطي قراءات غير صحيحة بدون إنذار.
- برمجيات خبيثة (Malware) تتحكم في النظام.

آليات التعامل مع الأعطال البيزنطية:

- استخدام التكرار والتنوع (Redundancy & Diversity).
- آليات الإجماع (Consensus Mechanisms) مثل خوارزمية Paxos أو Raft.
- التحقق من صحة المعلومات من مصادر متعددة.
- أنظمة كشف التسلل (Intrusion Detection Systems).

3.3 منحنى "حوض الاستحمام (Bathtub Curve)"

3.3.1 تعريف المنحنى

منحنى "حوض الاستحمام (Bathtub Curve)" هو نموذج رسومي يصف معدل الفشل خلال دورة حياة نظام مادي. يأخذ هذا المنحنى شكل حوض الاستحمام، حيث يكون معدل الفشل مرتفعاً في البداية، ثم ينخفض ويستقر، ثم يرتفع مرة أخرى في النهاية.

مراحل المنحنى:

السبب	معدل الفشل	الوصف	المرحلة
عيوب التصنيع، أخطاء التركيب، عيوب المواد	مرتفع (متناقص)	الفترة الأولى من عمر النظام	1. مرحلة الفشل المبكر (Infant Mortality)
الأعطال العشوائية، الظروف الخارجية	منخفض (ثابت تقريباً)	الفترة الوسطى من عمر النظام	2. مرحلة العمر الإنتاجي (Useful Life)
التآكل الطبيعي	مرتفع (متزايد)	الفترة الأخيرة من عمر النظام	3. مرحلة التآكل (Wear-Out)

المرجع المفصل للطلاب: وسائل تحقيق الوثوقية (Means to Attain Dependability)

تمهيد

بعد أن تعرفنا في الفصول السابقة على مفهوم الوثوقية، وركائزها الأساسية (الاعتمادية، التوفر، السلامة، النزاهة، السرية، قابلية الصيانة)، وكذلك على مهددات الوثوقية المتمثلة في الأعطال والأخطاء والفشل، يأتي دورنا الآن للإجابة على السؤال الأهم: كيف يمكننا تحقيق الوثوقية في أنظمة المعلومات؟

يجب هذا الفصل على هذا السؤال من خلال استعراض وسائل تحقيق الوثوقية (Means to Attain Dependability) الأربع الرئيسية، وهي استراتيجيات متكاملة تهدف إلى التعامل مع الأعطال في جميع مراحل دورة حياة النظام، بدءاً من التصميم وحتى التشغيل والصيانة.

أولاً: نظرة عامة على وسائل تحقيق الوثوقية

تُعرف وسائل تحقيق الوثوقية بأنها مجموعة من الاستراتيجيات والآليات التي يمكن تطبيقها للتعامل مع الأعطال، بهدف ضمان استمرارية تقديم الخدمة بشكل موثوق. وقد صنف الباحثون هذه الوسائل إلى أربع استراتيجيات رئيسية:

#	الوسيلة (بالعربية)	الوسيلة (بالإنجليزية)	الهدف الأساسي
1	تجنب الأعطال	Fault Avoidance	منع إدخال الأعطال أثناء التطوير
2	إزالة الأعطال	Fault Elimination	اكتشاف وإزالة الأعطال قبل التشغيل
3	تحمل الأعطال	Fault Tolerance	الاستمرار في تقديم الخدمة عند وجود أعطال
4	التنبؤ بالأعطال	Fault Forecasting	تقدير احتمالية وتأثير الأعطال المستقبلية

ملاحظة مهمة: هذه الاستراتيجيات ليست بديلة عن بعضها البعض، بل هي متكاملة وتُطبق جميعها في الأنظمة الموثوقة. فالنظام الذي يعتمد على استراتيجية واحدة فقط (مثل تحمل الأعطال دون تجنبها) سيكون مكلفاً وغير فعال.

ثانياً: تجنب الأعطال (Fault Avoidance)

2.1 التعريف والمفهوم

تجنب الأعطال (Fault Avoidance) هو مجموعة من الممارسات والتقنيات التي تهدف إلى منع إدخال الأعطال إلى النظام أثناء مراحل التطوير المختلفة (التخطيط، التحليل، التصميم، البرمجة). الفكرة الأساسية هي أن منع العطل من الأساس أفضل وأقل تكلفة من محاولة إصلاحه لاحقاً.

2.2 مبدأ أساسي: "الوقاية خير من العلاج"

في هندسة البرمجيات، ثبت أن تكلفة إصلاح العطل تزداد بشكل كبير كلما تأخر اكتشافه في دورة حياة التطوير:

مرحلة الاكتشاف	التكلفة النسبية للإصلاح
أثناء مرحلة تحليل المتطلبات	1x (الأقل تكلفة)
أثناء مرحلة التصميم	3x - 5x
أثناء مرحلة البرمجة	5x - 10x
أثناء مرحلة الاختبار	10x - 50x
بعد التسليم والتشغيل	50x - 200x (الأعلى تكلفة)

هذا الجدول يوضح لماذا يُعتبر تجنب الأعطال من أكثر الاستراتيجيات فعالية من حيث التكلفة.

2.3 آليات وأساليب تجنب الأعطال

أ. استخدام لغات برمجة آمنة (Safe Programming Languages)

بعض لغات البرمجة صُممت لتكون أكثر أماناً من غيرها، حيث تقلل من احتمالية ارتكاب المبرمج لأخطاء شائعة:

الميزات الأمنية	اللغة
نظام ملكية (Ownership) يمنع أخطاء الذاكرة؛ لا يوجد مؤشرات فارغة (Null Pointers)	Rust
لغة صممت خصيصاً للأنظمة الحرجة؛ تدعم العقود والتحقق القوي من الأنواع	Ada
إدارة تلقائية للذاكرة (Garbage Collection) تمنع تسرب الذاكرة	Java
قواعد نحوية بسيطة تقلل من الأخطاء البرمجية	Python

مثال توضيحي: في لغة C، يمكن للمبرمج أن يكتب كوداً يسبب تجاوز سعة المخزن المؤقت (Buffer Overflow) دون أن يصدر المحول البرمجي أي تحذير. أما في لغة Rust، فإن نظام الملكية يمنع حدوث مثل هذه الأخطاء أثناء الترجمة.

ب. تطبيق معايير البرمجة (Coding Standards)

المعايير البرمجية هي مجموعة من القواعد والإرشادات التي يجب على فريق التطوير اتباعها عند كتابة الكود. تهدف إلى:

- تحسين قابلية قراءة الكود.
- تقليل الأخطاء الشائعة.
- توحيد أسلوب الكتابة بين أعضاء الفريق.

أمثلة على معايير برمجية شهيرة:

- MISRA C/C++ تستخدم في الصناعات الحرجة (السيارات، الطيران).
- Google Java Style Guide: معيار لغة Java من جوجل.
- PEP 8: معيار كتابة كود Python.

مثال على قاعدة من: MISRA C

"يجب ألا يحتوي التعبير الشرطي في جملة if على عملية إسناد (=) بل يجب استخدام عملية المقارنة (==) فقط".
هذه القاعدة تمنع خطأ شائعاً حيث يكتب المبرمج `if (x = 5)` بدلاً من `if (x == 5)`، مما يسبب خطأ منطقياً يصعب اكتشافه.

ج. تدقيق المواصفات (Requirements Validation)

قبل البدء في كتابة أي كود برمجي، يجب التأكد من صحة واكتمال ودقة متطلبات النظام. فالعطل في المواصفات يؤدي إلى عطل في التصميم ثم عطل في التنفيذ.

تقنيات تدقيق المواصفات:

- المراجعات (Reviews): قراءة المواصفات من قبل فريق مستقل للتحقق من صحتها.
- النمذجة: (Prototyping) بناء نموذج أولي للنظام للتأكد من فهم المتطلبات.
- التتبع (Traceability): تتبع كل متطلب من مصدره إلى تنفيذه في الكود.

د. استخدام النماذج الرسمية (Formal Methods)

النماذج الرسمية هي تقنيات رياضية لوصف وتحليل أنظمة البرمجيات. باستخدامها، يمكن إثبات أن النظام يحقق خواص معينة رياضياً.

أمثلة على النماذج الرسمية:

- Z Notation: لغة رياضية لوصف الأنظمة.

- Alloy: أداة للتحليل النموذجي باستخدام المنطق الرياضي.
 - TLA+ (Temporal Logic of Actions): لغة لوصف الأنظمة المتزامنة والموزعة.
- الميزة: يمكن باستخدام النماذج الرسمية اكتشاف الأخطاء في التصميم قبل كتابة أي كود، مما يوفر الوقت والجهد.

ثالثاً: إزالة الأعطال (Fault Elimination)

3.1 التعريف والمفهوم

إزالة الأعطال (Fault Elimination) هي عملية اكتشاف وإزالة الأعطال الموجودة بالفعل في النظام قبل أن تؤدي إلى فشل. تركز هذه الاستراتيجية على تحسين جودة النظام عن طريق العثور على الأخطاء وإصلاحها.

3.2 الفرق بين تجنب الأعطال وإزالة الأعطال

إزالة الأعطال (Fault Elimination)	تجنب الأعطال (Fault Avoidance)
تكتشف وتزيل الأعطال الموجودة	تمنع الأعطال من الدخول إلى النظام
تُطبق أثناء التطوير وقبل التشغيل	تُطبق أثناء التطوير
تفاعلية (Reactive) ولكن قبل الفشل	استباقية (Proactive)
مثال: مراجعة الكود والاختبارات	مثال: معايير البرمجة

3.3 آليات وأساليب إزالة الأعطال

أ. مراجعة الكود (Code Review)

مراجعة الكود هي عملية قراءة الكود المصدري من قبل مطورين آخرين (غير كاتب الكود) للبحث عن الأخطاء والتحقق من جودته.

أنواع مراجعة الكود:

الميزة	الوصف	النوع
سريعة، مرنة	مراجعة سريعة بين زملاء الفريق	المراجعة غير الرسمية
أكثر دقة، تكشف أخطاء أكثر	عملية منظمة مع أدوار محددة (قائد، قارئ، كاتب)	التفتيش الرسمي (Inspection)
تتبع التغييرات وسهولة التعليق	استخدام أدوات مثل GitHub Pull Requests	المراجعة عن بُعد (Remote Review)

فوائد مراجعة الكود:

- اكتشاف الأخطاء المنطقية.
- تحسين تصميم الكود.
- نشر المعرفة بين أعضاء الفريق.
- ضمان الالتزام بمعايير البرمجة.

ب. التحليل الثابت (Static Analysis)

التحليل الثابت هو فحص الكود المصدري دون تنفيذه، باستخدام أدوات آلية للكشف عن الأخطاء والثغرات والانتهاكات للمعايير.

أمثلة على أدوات التحليل الثابت:

الوظيفة	اللغة	الأداة
تحليل جودة الكود، تغطية الاختبارات، الثغرات الأمنية	متعدد اللغات	SonarQube
اكتشاف الأخطاء النحوية والمنطقية	JavaScript	ESLint
تحليل جودة كود Python	Python	Pylint
اكتشاف الأخطاء الشائعة في Java	Java	FindBugs / SpotBugs
تحليل ثابت للغات C و C++	C/C++	Clang Static Analyzer

أنواع المشكلات التي يكشفها التحليل الثابت:

- المتغيرات غير المستخدمة.
- مؤشرات فارغة. (Null Pointer Dereference)
- تجاوز سعة المخزن المؤقت.
- الثغرات الأمنية مثل حقن. (SQL)
- انتهاكات معايير البرمجة.

مثال: أداة SonarQube يمكنها كشف أن هناك 12 عيباً خطيراً في مشروع برمجي، مع تحديد موقع كل عيب في الكود وشرح كيفية إصلاحه.

ج. الاختبارات (Testing)

الاختبارات هي عملية تنفيذ البرنامج بهدف اكتشاف الأخطاء. وهي من أهم آليات إزالة الأعطال.

أنواع الاختبارات (حسب المستوى):

1. اختبار الوحدة: (Unit Testing) اختبار أصغر وحدة برمجية (دالة، كلاس) بشكل منعزل. يُكتب عادةً من قبل المطور نفسه.
2. اختبار التكامل: (Integration Testing) اختبار تفاعل الوحدات المختلفة مع بعضها البعض، والتأكد من أنها تعمل معاً بشكل صحيح.
3. اختبار النظام: (System Testing) اختبار النظام المتكامل بأكمله للتأكد من مطابقته للمتطلبات.
4. اختبار القبول: (Acceptance Testing) اختبار النظام من قبل المستخدم النهائي للتأكد من أنه يلبي احتياجاته.

أنواع الاختبارات (حسب المنهجية):

1. اختبار الصندوق الأسود: (Black-box Testing) يعتمد على المواصفات دون النظر إلى الكود الداخلي. يركز على المدخلات والمخرجات.
2. اختبار الصندوق الأبيض: (White-box Testing) يعتمد على تحليل الكود الداخلي، ويهدف إلى تغطية جميع مسارات التنفيذ.
3. اختبار الصندوق الرمادي: (Gray-box Testing) يجمع بين الاثنين، حيث يكون لدى المختبر معرفة جزئية بالكود الداخلي.

مقاييس تغطية الاختبارات:

الوصف	المقياس
هل تم تنفيذ كل جملة برمجية في الكود؟	تغطية العبارات (Statement Coverage)

هل تم تنفيذ كل فرع من فروع الشرط (if/else) ؟	تغطية الفروع (Branch Coverage)
هل تم تنفيذ كل مسار ممكن في الكود؟	تغطية المسارات (Path Coverage)

د. إزالة الأعطال في الأنظمة المادية (Hardware Debugging)

في الأنظمة المدمجة (Embedded Systems) ، تتضمن إزالة الأعطال أيضاً أدوات مثل:

- محاكاة (Simulation): محاكاة النظام المادي قبل تصنيعه.
- تصحيح الأخطاء (Debugging): استخدام أدوات مثل JTAG لتصحيح الأخطاء في النظام المادي.
- اختبار الإجهاد (Stress Testing): اختبار النظام في ظل ظروف تشغيل قصوى.

رابعاً: تحمل الأعطال (Fault Tolerance)

4.1 التعريف والمفهوم

تحمل الأعطال (Fault Tolerance) هو قدرة النظام على الاستمرار في تقديم الخدمة بشكل صحيح حتى في وجود أعطال . بدلاً من محاولة منع أو إزالة الأعطال بشكل كامل، يعترف بحمل الأعطال بأن الأعطال ستحدث حتماً، ولذلك يصمم النظام ليكون قادراً على التعامل معها دون انقطاع الخدمة.

4.2 مبدأ أساسي: "التكرار هو جوهر تحمل الأعطال"

الفكرة الأساسية في تحمل الأعطال هي التكرار (Redundancy) ، وهو استخدام مكونات إضافية (أجهزة، برمجيات، بيانات) تؤدي نفس الوظيفة، بحيث إذا فشل أحدها، يتولى الآخر المهمة.

أنواع التكرار:

المثال	الوصف	النوع
خوادم احتياطية، أقراص RAID	وجود مكونات مادية متطابقة	التكرار المادي (Hardware Redundancy)
إصدارات مختلفة من نظام التحكم في الطيران	وجود إصدارات مختلفة من البرنامج تؤدي نفس الوظيفة	التكرار البرمجي (Software Redundancy)
إعادة إرسال حزمة بيانات في الشبكة	إعادة تنفيذ العملية في حال فشلت المرة الأولى	التكرار الزمني (Time Redundancy)
بتات التكاثر، المجاميع الاختبارية	إضافة بيانات إضافية للتحقق من الصحة	التكرار المعلوماتي (Information Redundancy)

4.3 آليات وأساليب تحمل الأعطال

أ. التكرار النشط (Active Redundancy) مقابل التكرار السلبي (Passive Redundancy)

التكرار السلبي (Passive)	التكرار النشط (Active)
مكون رئيسي يعمل، والآخر في وضع الاستعداد	جميع المكونات تعمل في نفس الوقت
يتم التبديل تلقائياً عند اكتشاف عطل	يتم اتخاذ القرار بالإجماع (Voting)
زمن استجابة = زمن التبديل	زمن استجابة فوري (لا يوجد تأخير)
مثال: خادم احتياطي (Backup Server)	مثال: أنظمة الطيران (ثلاثة أجهزة كمبيوتر تصوت)

ب. أنظمة التصويت (Voting Systems)

في الأنظمة ذات التكرار النشط، يتم استخدام آلية التصويت لتحديد النتيجة الصحيحة عند وجود اختلاف بين المكونات. أنظمة التصويت الشائعة:

الوصف	النظام
ثلاثة مكونات تؤدي نفس العملية، ويتم اعتماد نتيجة الأغلبية (2 من 3)	TMR (Triple Modular Redundancy)
خمس مكونات، وتُعتمد نتيجة الأغلبية (3 من 5)	5MR (5 Modular Redundancy)

مثال: في مكوك الفضاء، استُخدم نظام TMR مع ثلاثة أجهزة كمبيوتر متطابقة. إذا اختلفت نتائجها، يُعتمد نتيجة الجهازين المتطابقين.

ج. آليات الكشف عن الأعطال واكتشافها (Fault Detection)

لكي يتمكن النظام من التعامل مع العطل، يجب أولاً أن **يكتشف وجوده**. من آليات الكشف الشائعة:

1. **المجمام الاختبارية (Checksums):** حساب قيمة رياضية من البيانات، وإعادة حسابها للتحقق من عدم التغيير.
2. **التوقيعات الرقمية (Digital Signatures):** التحقق من أن البيانات لم يتم التلاعب بها.
3. **المؤقتات (Watchdog Timers):** مؤقتات تراقب تنفيذ العمليات، وتنبه في حال توقفها.
4. **اختبارات الصحة (Health Checks):** اختبارات دورية للتأكد من أن المكونات تعمل بشكل صحيح.
5. **الكشف عن التوقيت (Timing Detection):** مراقبة توقيت العمليات، فإذا تجاوزت زمناً محدداً، اعتبر أنها فشلت.

د. آليات التعافي من الأعطال (Recovery Mechanisms)

بعد اكتشاف العطل، يجب أن يتعافى النظام ويعود إلى الحالة الصحيحة:

المثال	الوصف	الآلية
استعادة قاعدة البيانات من نسخة احتياطية	العودة إلى حالة سابقة صحيحة (Checkpoint)	التراجع (Rollback)
إعادة إرسال طلب فشل بسبب مشكلة شبكة	إعادة تنفيذ العملية مع تجاهل العطل	التقدم للأمام (Rollforward)
إعادة تشغيل خدمة متوقفة في السحابة	إعادة تشغيل المكون المتعطل	إعادة التشغيل (Restart)
عزل خادم مخترق عن الشبكة	فصل المكون المتعطل عن باقي النظام	العزل (Isolation)

هـ. آليات التعامل مع الأعطال البيزنطية

الأعطال البيزنطية (حيث يتصرف المكون بشكل خبيث أو عشوائي) تتطلب آليات خاصة، مثل:

1. **خوارزمية الإجماع البيزنطي (Byzantine Fault Tolerance - BFT):** مثل خوارزمية PBFT المستخدمة في العملات الرقمية.

2. **التوقيعات الرقمية:** للتحقق من صحة الرسائل ومنع الانتحال.

3. **التنوع (Diversity):** استخدام مكونات مختلفة التصميم، بحيث لا يقع جميعها في نفس العطل.

مثال: نظام التصويت بأغلبية 3/2 (أي 2 من 3) يتحمل عطلاً بيزنطياً واحداً، بينما نظام 3 من 5 يتحمل عطلين بيزنطيين.

و. تحمل الأعطال في الأنظمة الموزعة

الأنظمة الموزعة (مثل الخدمات السحابية) تواجه تحديات إضافية في تحمل الأعطال:

آلية التعامل	التحدي
--------------	--------

استخدام نسخ احتياطية (Replicas)	فشل العقدة (Node Failure)
خوارزميات الإجماع (Paxos) ، Raft	انقطاع الشبكة (Network Partition)
استخدام المهلات (Timeouts) وإعادة المحاولة	تأخر الشبكة (Network Latency)
التوزيع الجغرافي (Geo-Redundancy)	فشل المركز البيانات (Datacenter Failure)

خامساً: التنبؤ بالأعطال (Fault Forecasting)

5.1 التعريف والمفهوم

التنبؤ بالأعطال (Fault Forecasting) هو عملية تقدير احتمالية حدوث الأعطال المستقبلية وتأثيرها على النظام. تهدف هذه الاستراتيجية إلى توفير معلومات كمية تساعد في اتخاذ قرارات هندسية وإدارية حول كيفية تحسين الوثوقية.

5.2 الفرق بين التنبؤ والوسائل الأخرى

التنبؤ بالأعطال	تحمل الأعطال	إزالة الأعطال	تجنب الأعطال
تتنبأ بالأعطال المستقبلية	تتعامل مع الأعطال عند حدوثها	تزيل الأعطال الموجودة	تمنع الأعطال
استباقية (تحليلية)	تفاعلية (في وقت التشغيل)	تفاعلية	استباقية
قبل وأثناء التشغيل	أثناء التشغيل	قبل التشغيل	أثناء التطوير

5.3 آليات وأساليب التنبؤ بالأعطال

أ. النمذجة الإحصائية (Statistical Modeling)

تُستخدم النماذج الإحصائية للتنبؤ بمعدلات الفشل باستخدام بيانات تاريخية.

أمثلة على النماذج:

1. نموذج التوزيع الأسّي (Exponential Distribution): يفترض معدل فشل ثابتاً (λ) ، ويُستخدم عندما تكون الأعطال عشوائية.

2. نموذج ويبيل (Weibull Distribution): نموذج مرن يمكنه تمثيل معدلات فشل متزايدة أو متناقصة أو ثابتة، ويُستخدم في تحليل أعطال التدهور.

3. منحنى حوض الاستحمام (Bathtub Curve): نموذج شهير يصف معدل الفشل خلال دورة حياة النظام. تطبيق عملي: يمكن استخدام نموذج ويبيل لتقدير أن قرصاً صلباً لديه احتمال فشل 5% خلال السنة الأولى، و 15% خلال السنة الثالثة.

ب. تحليل شجرة الأعطال (Fault Tree Analysis - FTA)

تحليل شجرة الأعطال هو أسلوب استنتاجي (Top-down) يبدأ من حدث غير مرغوب (الفشل) ويحلل الأسباب التي قد تؤدي إليه.

خطوات FTA:

1. تحديد حدث القمة - (Top Event) الفشل الذي نريد تحليله.
2. تحديد الأحداث الأساسية التي قد تسبب هذا الفشل.
3. ربط الأحداث باستخدام بوابات منطقية (AND ، OR ، NOT).
4. حساب احتمالية الحدث بناءً على احتمالية الأحداث الأساسية.

مثال: تحليل شجرة الأعطال لفشل نظام الفرامل في السيارة:

text

فشل نظام الفرامل



ج. تحليل تأثير وأنماط الأعطال (Failure Mode and Effects Analysis - FMEA)

تحليل FMEA هو أسلوب استقرائي (Bottom-up) يبدأ من المكونات الفردية ويحلل تأثير فشل كل مكون على النظام ككل.

خطوات FMEA:

1. تحديد جميع مكونات النظام.
2. تحديد أنماط الفشل المحتملة لكل مكون.
3. تقييم تأثير كل فشل على النظام.
4. تصنيف المخاطر حسب الشدة والاحتمالية وقابلية الكشف.
5. اقتراح إجراءات تخفيفية للمخاطر العالية.

مقياس تصنيف المخاطر في FMEA:

المقياس	الدرجة (1-10)	الوصف
الشدة (Severity - S)	1-10	مدى خطورة تأثير الفشل
الاحتمالية (Occurrence - O)	1-10	مدى احتمالية حدوث الفشل
قابلية الكشف (Detection - D)	1-10	مدى سهولة اكتشاف الفشل قبل حدوثه

حساب أولوية المخاطر: (RPN - Risk Priority Number)

$$RPN = S \times O \times D$$

كلما زاد RPN ، زادت أولوية معالجة هذا الفشل.

د. تحليل المخاطر (Risk Analysis)

تحليل المخاطر هو عملية أوسع تشمل تحديد وتقييم وتخفيف المخاطر التي قد تؤثر على النظام.

أنواع المخاطر:

المثال	الوصف	نوع المخاطرة
عدم توافق المكونات، تقنية غير مثبتة	تتعلق بالتكنولوجيا المستخدمة	مخاطر تقنية
تأخير الجدول الزمني، تجاوز الميزانية	تتعلق بإدارة المشروع	مخاطر إدارية
هجمات القرصنة، تسرب البيانات	تتعلق بأمن النظام	مخاطر أمنية
فشل في الصيانة، خطأ بشري	تتعلق بتشغيل النظام	مخاطر تشغيلية

خطوات تحليل المخاطر:

1. **تحديد المخاطر:** ما هي الأشياء التي قد تسوء؟
2. **تحليل المخاطر:** ما هو احتمال حدوثها؟ وما هو تأثيرها؟
3. **تقييم المخاطر:** ما هي المخاطر التي يجب معالجتها أولاً؟
4. **تخفيف المخاطر:** ما هي الإجراءات التي ستتخذ للتعامل مع كل خطر؟

سادساً: تطبيق متكامل لوسائل تحقيق الوثوقية

6.1 خريطة طريق لتحقيق الوثوقية في مشروع برمجي

يمكن تلخيص التطبيق المتكامل لوسائل تحقيق الوثوقية في دورة حياة المشروع كما يلي:

مرحلة المشروع	وسائل تحقيق الوثوقية المطبقة
تحليل المتطلبات	تدقيق المواصفات، النماذج الرسمية، تحليل المخاطر (FTA) ، FMEA
التصميم	التصميم الآمن، التكرار (Redundancy) ، آليات تحمل الأعطال
التطوير (البرمجة)	معايير البرمجة، لغات آمنة، مراجعة الكود، التحليل الثابت
الاختبار	اختبار الوحدات، التكامل، النظام، القبول، اختبار الإجهاد
النشر والتشغيل	مراقبة الأداء، أنظمة الكشف عن الأعطال، أنظمة التعافي
الصيانة والتحديث	التحديثات الأمنية، مراقبة المقاييس، التنبؤ بالأعطال

6.2 مثال تطبيقي شامل: نظام بنكي لإدارة الحسابات

لنفترض أننا نطور نظاماً بنكياً لإدارة حسابات العملاء، كيف نطبق وسائل تحقيق الوثوقية الأربع؟

الوسيلة	التطبيق في النظام البنكي
تجنب الأعطال	استخدام لغة برمجة آمنة (Java) ، معايير برمجية صارمة، تدقيق المتطلبات مع البنك
إزالة الأعطال	مراجعة الكود بين المطورين، استخدام SonarQube للتحليل الثابت، اختبارات شاملة (وحدة، تكامل، نظام)
تحمل الأعطال	خوادم احتياطية، نسخ احتياطية لقاعدة البيانات (Replication) ، نظام تصويت للعمليات الحرجة، آليات Rollback في حال فشل المعاملة
التنبؤ بالأعطال	مراقبة أداء النظام، تحليل سجلات الأخطاء، استخدام نماذج إحصائية لتوقع فشل الأقراص، تحليل FMEA للمكونات الحرجة

6.3 التحديات في تطبيق وسائل تحقيق الوثوقية

رغم أهمية هذه الوسائل، إلا أن تطبيقها يواجه تحديات:

الوصف	التحدي
بعض الوسائل (مثل التكرار، النماذج الرسمية) مكلفة وتتطلب وقتاً وجهداً إضافياً	التكلفة
إضافة آليات تحمل الأعطال تزيد من تعقيد النظام	التعقيد
تحسين وثوقية قد يؤثر سلباً على أداء أو تكلفة النظام	المقايضات (Trade-offs)
تتطلب بعض التقنيات (مثل النماذج الرسمية) مهارات متخصصة	المهارات
دمج جميع الوسائل في نظام واحد يتطلب دقة	التكامل

خلاصة الفصل

في هذا الفصل، تعرفنا على وسائل تحقيق الوثوقية الأربع الرئيسية، وهي استراتيجيات متكاملة تهدف إلى ضمان استمرارية النظام وتقديم خدمة موثوقة:

1. **تجنب الأعطال: (Fault Avoidance)** منع الأعطال من الدخول إلى النظام باستخدام لغات أمنة، معايير برمجية، وتدقيق المواصفات.
2. **إزالة الأعطال: (Fault Elimination)** اكتشاف وإزالة الأعطال الموجودة قبل التشغيل عبر المراجعات، التحليل الثابت، والاختبارات.
3. **تحمل الأعطال: (Fault Tolerance)** الاستمرار في تقديم الخدمة عند وجود أعطال باستخدام التكرار، التصويت، وآليات التعافي.
4. **التنبؤ بالأعطال: (Fault Forecasting)** تقدير احتمالية وتأثير الأعطال المستقبلية باستخدام النماذج الإحصائية، FMEA، و FTA

الرسالة الأساسية: لا توجد وسيلة واحدة كافية لتحقيق الوثوقية. النظم الموثوقة حقاً هي تلك التي تدمج جميع هذه الوسائل في تصميمها وتشغيلها، مع مراعاة التحديات والمقايضات المرتبطة بكل منها.

أسئلة للمراجعة والتفكير

1. لماذا تعتبر الوقاية (تجنب الأعطال) أفضل من العلاج (إزالة الأعطال) من حيث التكلفة؟
2. قارن بين التكرار النشط والتكرار السلبي في تحمل الأعطال، مع إعطاء مثالين تطبيين.
3. ما هي مزايا وعيوب استخدام النماذج الرسمية في تجنب الأعطال؟
4. كيف يمكن تطبيق تحليل FMEA على نظام مراقبة جوي؟ اذكر ثلاثة أنماط فشل محتملة لكل مكون رئيسي.
5. اختر نظاماً برمجياً تعرفه، وشرح كيف يمكن تطبيق وسائل تحقيق الوثوقية الأربع عليه.
6. ما هي المقايضة بين تحمل الأعطال والأداء في الأنظمة في الزمن الحقيقي (Real-time Systems) ؟
7. كيف تساهم مراجعة الكود في كل من تجنب الأعطال وإزالة الأعطال؟